

Sprint: Feature Grounding Engine

Branch: feature-grounding-engine • **Requirement under test:** Add Employee (only) • **Discipline:** one defect, reproduced under Gate 0, fixed and proven before/after — no new engines, prompts, LLM calls, or architecture.

Defect statement (the ONE thing this sprint fixes):

Scenarios belonging to the same feature were **independently grounded**, causing **inconsistent form selection**. Some scenarios of the Add Employee feature got concrete steps on the right form; others — sharing no vocabulary with the captured field labels — fell to placeholders; and file/authorization scenarios could match a foreign search-filter form and inherit its junk labels.

Acceptance criterion added by the founder:

Classify each scenario by its **INTENT, not its title**. Form-entry scenarios ground on the resolved feature form; authorization / authentication / session / direct-URL scenarios are **held** — they must NOT be given fabricated form-fill steps.

Core product rule (non-negotiable): Never generate confident but incorrect test artifacts. We do not improve one metric by silently introducing a new class of wrong output.

Gate 0 — mandatory reproduction before any code changed

Nothing entered the sprint without a decisive reproduction. The uploaded evidence (test-cases-233.csv : 35 Add-Employee cases, 23 placeholder steps "Exercise the X scenario", Automation Ready 34%) came from an older build, so I did **not** trust it at face value. I built a faithful OrangeHRM **mixed profile** (an Add Employee form **and** an Employee-Search filter form, multiple pages, shared vocabulary) and swept the one variable I could not know from the CSV alone: whether the crawler captured the incidental token "employee" on the Add form.

Gate 0 variant	Placeholder	Wrong-form / search leak	Automation Ready
V1 — Add form with the "employee" token + search form	0	0	100%
V2 — Add form without the "employee" token + search form	25 (71%)	3 (9%)	29%
V3 — search form only (Add form not captured)	32	present	9%

The smoking gun: V2 reproduces the uploaded CSV almost exactly (25 vs 23 placeholders, 29% vs 34% AR). The only difference between the 100%-clean V1 and the broken V2 is an **incidental crawl artifact** — whether one field happened to be labelled “Employee Id”. Automation readiness was hinging on a coincidence of vocabulary, not on scenario quality. That confirmed a **live, feature-level form-resolution defect** (Branch A), not a resilience fallback — so the fix was worth a sprint.

Root cause

File: `src/engines/scenario-builder.ts` • **Function:** `buildDraftTestCases()`

The builder called `pickForm(ap?.forms, featureTerms)` **inside the per-scenario loop**, using each scenario’s **narrow** vocabulary (its own title + objective + risk area). Because scenarios of the same feature carry very different wording:

- A scenario like **“SQL-injection input is rejected”** or **“Duplicate entry is handled”** shares no token with the captured field labels (First Name / Last Name / ...) → scored **0** → `pickForm` returned `undefined` → the scenario fell to a **placeholder**.
- A **file-upload** or **unauthorized-access** scenario could score against the foreign Employee-Search filter form and inherit its junk labels (“Type for hints...”, “comma separated words...”).

So the **same feature** resolved **different forms — or none — scenario by scenario**. The form’s identity is a property of the feature, but it was being decided from the accidental wording of each individual scenario.

The fix — two deterministic, data-only parts (no new engine, no LLM)

Part 1 — Feature-level form resolution (resolve ONCE, reuse)

The feature form is now resolved a **single time**, from the feature’s pooled vocabulary (requirement title ∪ the union of every **form-interacting** scenario’s terms), and reused by every scenario that fills it:

```
const featureVocab = Array.from(new Set([...titleTerms, ...groundedScenarioTerms]));
const featureForm = pickForm(ap?.forms, featureVocab); // resolved ONCE, before the loop
```

Because the pool now contains the strong field-identity tokens contributed by the create/field scenarios (First Name, Last Name, Photo, ...), the correct Add form wins for **every** scenario — including the injection / duplicate / whitespace scenarios that used to score 0 on their own. The anti-leak guarantee from the earlier trust-bug fix is preserved unchanged: `pickForm` still returns `undefined` when **nothing** matches, so an unrelated form is never grounded on.

Part 2 — Intent-based grounding + honest hold (classify by intent, not title)

A pure, deterministic classifier reads each scenario’s **stable structured signals** — its canonical `id`, its `riskArea`, and the KB-declared `stepFlow` — and **never its title**:

```
export function classifyGroundingIntent(s): ScenarioGroundingIntent
//   'authorization' | 'file_upload' | 'search' | 'navigation' | 'form_entry'
```

- `form_entry` / `file_upload` / `search` / `navigation` → **ground on the resolved feature form**. (File fields are uploaded, not typed into: the step reads `Upload ...` for the Profile Photo .)
- `authorization` (authorization / authentication / session / direct-URL) → **HELD**. These are not form interactions — their real steps (requesting a URL without a session, acting without the required role) belong to the future Intent-aware Step Generator. Rather than fabricate Add-form fill steps, the builder emits an honest skeleton and marks the case **Needs Review** with a precise reason.

Classifying by intent (not title) is what makes the hold correct: a scenario titled “Created record is immediately searchable” is still a **form_entry** (it creates a record), while “Unauthorized user cannot perform the operation” is **authorization** (held) — the title never decides.

A trust-regression caught and closed during the sprint

Pooling every scenario’s terms initially re-opened the original trust bug: the held authorization scenario “Unauthenticated user is redirected to login” voted the word “**login**” into the feature vocab, which then matched a login form and poured `Username / Password` steps into Add-Employee cases. The fix is principled and is part of this sprint: **the feature vocabulary is drawn only from the scenarios that actually fill the form — held (authorization) scenarios never influence which form the form scenarios ground on**. This is locked by the existing trust test (`scenario-correctness`) plus the new suite.

Before / After — the founder’s success criteria (proven)

Measured by running the current builder on the **exact V2 mixed profile** that reproduced the defect (35 scenarios, default families):

Metric	Before (V2 repro)	After	Target
Placeholder (form scenarios)	25	0	0
Wrong-form leakage	3	0	0
Search-widget leakage	present	0	0
Automation Ready	29%	91% (100% of the 32 form scenarios)	> 85%
Auth scenarios wrongly converted to form-fill	0	0 — all 3 remain Needs Review	0

The most important row is the last one. The three authorization scenarios — `crud-neg-direct-endpoint-authz` , `crud-neg-unauthenticated-redirect` , `crud-neg-unauthorized` — are **held as Needs Review with zero fabricated form steps**. We reached 0 placeholders and

>85% automation-ready **without** inventing a single confident-but-incorrect authorization case. That is the whole point: the placeholder metric did not improve by quietly manufacturing wrong artifacts elsewhere.

Verification

- `npx tsc --noEmit` — clean.
 - **New suite** `tests/unit/feature-grounding-engine.test.ts` — **12 tests**, covering the intent classifier (each intent resolved from stable signals; title never used) and end-to-end feature-level resolution (all form scenarios grounded on the one Add form; zero search leak; file fields uploaded; every authorization scenario held with a precise reason and 0 wrongly converted).
 - **Regression** — all green, run individually: `scenario-builder` (28), `scenario-correctness` (25), `scenario-integrity` (32), `qa-knowledge-stepflow` (4), `scenario-planner` (53), `scenario-planner-field-aware` (5), `scenario-planner-standard-coverage` (7), `scenario-graph` (22), `scenario-renderer` (14), `canonical-validator` (11), `qa-standard-validator` (17), `expected-result-validator` (26), `expected-result-excellence` (21), `requirement-coverage-engine` (12), plus the standalone check scripts (`canonical-test-case` 35, `dedup-scenario-aware` 7, `automationexercise-grounding` 18, `three-flows-deterministic` 24, `test-case-coverage` 23). The one pre-existing `credential-grounding` failure is unrelated (script-gen password env fallback) and fails identically on the clean tree.
-

Scope discipline — what this sprint deliberately did NOT do

- **Generation stays AI; grounding stays deterministic.** The classifier and feature-level resolver read only data that already exists (canonical id, risk area, KB stepFlow, App-Profile forms). No new engine, prompt, inference, or LLM call was added.
- **Authorization/session steps are held, not faked.** Giving those scenarios their real, intent-specific steps is **Sprint 2 (Intent-aware Step Generation)** — the natural next sprint — not this one.
- **Not touched:** planner fragmentation (`req-step-*`), token-optimization architecture, entity normalization, scenario-specific test data. All remain on the roadmap/backlog and were left exactly as-is.

Bar for done: a QA Lead opens a generated Add-Employee case and can execute it without rewriting it — while the authorization cases honestly announce themselves as Needs Review instead of pretending to be automatable.